

The Four Tenants

A literal, click-by-click walkthrough of every real action for Tenants 1–4 — from opening a browser at the sign-up page through three simulated months of paid use. Every tool used is named and explained before it's needed, every step says exactly what to click and what to type, and every step says exactly how to tell it worked from how to tell it didn't.

Companion to

Five-Tenant Readiness Dossier

Covers

Phases 1–4, complete

Written

2026-08-27

HOW TO READ THIS GUIDE

Each numbered step is tagged **YOU – click this yourself** or **TECH – engineer sets this up**, and names the exact tool it uses. A few real-world situations — a partial M-Pesa payment, a payment that never confirms, three months of history — can't be produced by clicking buttons in real time, so those are staged directly against the database by the engineer, with the exact tool and exact values spelled out at each step.

IN THIS GUIDE

1. Before you start
2. The tools used in this guide
3. Chrome DevTools, from zero
4. The admin console, from zero
5. Prisma Studio, from zero
6. Tenant 1 — WhatsApp Only → Starter
7. Tenant 2 — Messenger Only → Professional
8. Tenant 3 — Telegram Only → Business
9. Tenant 4 — Widget Only → Enterprise
10. Payment edge cases
11. Three simulated months
12. Result log

COVERAGE MAP

WHAT'S BEING VERIFIED	COVERED IN
Sign-up, OTP verification, trial creation	Tenants 1-4, "Sign up" step group
Every channel type connected at least once	Tenants 1-4, "Connect" step groups (WhatsApp, Messenger, Telegram, Widget)
Every paid tier reached at least once	Tenants 1-4, "Upgrade" step groups (Starter, Professional, Business, Enterprise)
Full payment, partial payment, delayed settlement, failed payment	Payment edge cases
Usage exhausting a real cap, and resetting on schedule	Three simulated months
Dashboard correctness, per tenant	Every "Verify before moving on" box
Admin-side correctness, per tenant	Every "Verify before moving on" box; Three simulated months
Profit margin and money reconciliation	Three simulated months
Audit trail completeness	Three simulated months

Ground rules

Read once, applies to every tenant below.

WHAT YOU'LL NEED

- A browser (any modern one), pointed at `https://p2less-app-production.up.railway.app`.
- Four throwaway test identities — a name, an `@example.com`-style email, and an obviously-fake phone number for each tenant. Suggested pattern: `+254700111001` for Tenant 1, `...002` for Tenant 2, and so on.
- An admin login for whoever plays the "Tech" role in the steps marked that way.
- Nothing else. No AI provider usage happens anywhere in this guide — every reply path used below is deterministic (menu numbers, OTP codes, templated lookups), never a real AI call.

WHY THE OTP CODE SHOWS ON SCREEN

Production doesn't have a real SMS provider configured yet, so the sign-up flow honestly labels this "Demo only" and prints the real 6-digit code directly on the page instead of texting it. This is the actual, current, honest behavior of the app — not a workaround for this test.

Every tool used in this guide, explained first

Unlike the abuser's manual, this guide mostly needs nothing but a browser. The few extra tools are only for the "Tech" steps, and each is explained fully, from zero, in the three short chapters right after this one.

Chrome (or any browser)

EVERYTHING A CUSTOMER SEES

The only tool needed for every step tagged "You" — sign-up, channel connection, upgrades, payment.

Used for: essentially this whole guide.

Chrome DevTools OPTIONAL, FOR READING RESPONSES

A panel built into Chrome (**F12**). Lets you see the exact reply the server sent back for any action, not just what the page renders.

Used for: double-checking a payment or invoice response when the on-screen message alone isn't enough to be sure.

The P2Less admin console TECH ROLE

P2Less's own internal dashboard for staff — tenant list, billing records, platform-wide usage. A real part of the product, not a testing tool.

Used for: every admin-side correctness check, and the one Enterprise plan change that's deliberately admin-only.

Prisma Studio TECH ROLE

A free, official, local tool that comes with this project's database layer (Prisma). Running one command opens a web page showing every table as an editable spreadsheet.

Used for: the handful of things no amount of clicking can produce — a partial payment, a late payment, three months of dated history.

Chrome DevTools, from zero

You won't need this for most of the guide — the on-screen messages are usually enough. It's here for the moments where you want to see the real, underlying server response with your own eyes.

1 You — DevTools

With Chrome open on any page, press **F12**. (If nothing happens, right-click the page → **Inspect**.)

Expect: a panel opens with tabs across the top — Elements, Console, Network, Application, and more.

2 You — DevTools

Click **Network**, then tick **Preserve log** so the list doesn't clear as you navigate.

3 You — DevTools

Click any row in the list to see its **Status** code and its **Response** tab — the server's actual reply. A **200** means it succeeded; a **400** / **401** / **403** means it was refused; a **500** means the server itself broke. The Response tab often has a short, readable message explaining exactly which of those happened and why.

The admin console, from zero

Whoever plays the Tech role for admin-side checks should be comfortable here before Tenant 1 even signs up.

1 Tech

Log into P2Less as a platform admin, then go to `/admin/tenants`.

Expect: a list of every tenant on the platform, each row showing its name, plan, and status at a glance.

2 Tech

Click any tenant's row.

Expect: that tenant's own detail page — plan, subscription status, owner contact, and (depending on what's built) recent activity.

3 Tech

Go to `/admin/billing`.

Expect: a platform-wide list of invoices and payments, each tied back to a tenant, with amounts and statuses.

4 Tech

Note the URL pattern for each — you'll be asked to revisit these exact pages, filtered to a specific tenant, throughout this guide.

Prisma Studio, from zero

The one tool this guide asks an engineer to actually install and run. It's already part of the project — nothing new to download.

1 Tech

Open a terminal in the project's folder (`p2less-platform`) and run `npx prisma studio` .

Expect: a message that it's running, and a browser tab opens automatically at `localhost:5555` .

2 Tech

On the left, you'll see every table in the database — `Tenant` , `Subscription` , `Payment` , `Invoice` , `UsageEvent` , `AiRequestLog` , and more. Click any one to see its rows as an editable spreadsheet.

3 Tech

To add a row: click **Add record** (usually top-right), fill in the fields shown, click **Save 1 change**. To edit one: click directly into any cell, change it, save the same way.

4 Tech

Leave this tab open for the whole guide — you'll come back to it for every step tagged "Tech."

BE CAREFUL WITH THIS TOOL

It has full read/write access to the real production-shaped database you point it at. Every "Tech" step in this guide tells you exactly which table and which fields to touch — stick to those, and double-check you're pointed at the intended database (test/staging, not a real customer's data) before saving anything.

WhatsApp Only → Starter

A small business signing up, connecting WhatsApp, and upgrading to the cheapest paid tier.

Sign up

1 YOU — BROWSER

Open a browser and go to `p2less-app-production.up.railway.app/onboard`.

Expect: a page titled "Your conversational platform, ready in minutes," with a 3-step outline (Tell us about your organization · Verify your phone · Start exploring).

2 YOU — BROWSER

Scroll to the form. In **Organization name**, type `Tenant 1 — WhatsApp Only`. Leave Industry as **Business**.

3 YOU — BROWSER

In **Your phone number**, type your Tenant 1 test number, e.g. `+254700111001`.

4 YOU — BROWSER

Fill **Your name** and **Your email** with your Tenant 1 test identity, then click **Create my workspace**.

Expect: the page advances to "Verify your phone number," with an orange "Demo only" box showing a real 6-digit code.

5 YOU — BROWSER

Type that exact code into the **6-digit code** field and click **Verify and create workspace**. Wait for the page to finish — don't navigate away.

How to know it worked: you land signed-in, on either `/explore` or the real dashboard, with a visible "Welcome" heading. If instead you're bounced back to the OTP screen, the code was mistyped or already used — reload `/onboard` and start over from a clean state rather than retrying the same code.

Check the trial is real

1 YOU — BROWSER

Go to `/dashboard`. Look at the card titled **Free trial balance** near the top.

Expect: a badge reading "7 days left" (or close to it), and two progress bars both showing `0 / 200` and `0 / 100`.

TECH — PRISMA STUDIO

In the already-open Prisma Studio tab, click the `Subscription` table, find Tenant 1's row (match it by its `tenantId`, cross-referenced from the `Tenant` table by name), and look at its `trialEndsAt` column.

How to know it's right: a real date roughly 7 days from today, not blank/null.

Connect WhatsApp

1 YOU — BROWSER

In the left sidebar, click **Channels**.

2 YOU — BROWSER

Under the WhatsApp section, click **Connect via alternative** (the camera-free, typed-pairing-code path is the most reliable one to test with).

3 YOU — BROWSER

Optionally type a real WhatsApp number of your own into "Camera not working? Enter your WhatsApp number," then submit.

How to know it worked: a "Scan to connect" modal opens showing either a QR code or a typed pairing code, and stays open (this exact modal not disappearing on its own was a real bug found and fixed earlier this week — worth specifically confirming it still holds by waiting 15–20 seconds without clicking anything and checking it hasn't vanished).

4 YOU — BROWSER

If you have a real phone free to pair, finish linking it. If not, close the modal and note that connection was left pending — that's a legitimate, logged outcome, not a failure of this step.

Upgrade to Starter

1 YOU — BROWSER

Go to `/dashboard/billing`. Find the **Starter** row and click **Upgrade**.

Expect: a modal opens, briefly says "Calculating your upgrade...", then shows a real invoice: Invoice number, Current plan (Trial allowance), Current plan value KES 0, Starter plan price KES 1,500, Amount payable KES 1,500.

2 YOU — BROWSER

Click **M-Pesa STK Push**, enter a real or test phone number, click **Pay KES 1,500**.

Expect (demo mode): "Recorded (demo mode)... Your upgrade is now active" almost immediately, since no real Daraja keys are configured. **Expect (if Daraja IS configured):** a real STK prompt on the phone — approve it, then wait for the page to detect confirmation.

3 YOU — BROWSER

Once the modal shows "Payment confirmed," click **Done**, then reload `/dashboard`.

How to know it worked: the subtitle under "Welcome" now reads "... · Starter plan," and the usage card now shows a real KES balance instead of the trial progress bars. If the subtitle still says trial after a reload, the payment didn't actually settle — check DevTools' Network tab response for the payment request for a clearer reason.

VERIFY BEFORE MOVING ON

- **Tech — Admin console:** go to `/admin/tenants`, find Tenant 1's row. **Expect:** plan column reads "Starter," status "active," owner email matches what you signed up with.
- **Tech — Admin console:** go to `/admin/billing`, filter or search for Tenant 1. **Expect:** one real payment of KES 1,500 against this tenant, status "paid" or equivalent.
- **You — Browser:** reload the Channels page. **Expect:** WhatsApp's real connection status shown accurately — connected, pending, or not connected, matching whatever you actually did above, never a stale or mismatched

badge.

Messenger Only → Professional

Same shape as Tenant 1, on Messenger, upgrading one tier higher.

Sign up & verify trial

1 YOU – BROWSER

Repeat the Tenant 1 sign-up steps exactly, using **Tenant 2 – Messenger Only** and your Tenant 2 test identity/phone (e.g. **+254700111002**).

2 YOU – BROWSER

Confirm the same "7 days left," 0/200, 0/100 usage card as Tenant 1.

Connect Messenger

1 YOU – BROWSER

Go to **Channels**, find the Facebook Messenger section, click **Connect a Facebook Page**.

Expect: Meta's real OAuth page opens if a test Facebook Page and app config are available.

2 YOU – BROWSER

If no real test Page is available, stop here and log it plainly: *"External dependency — no test Facebook Page available, Messenger connection not exercised."* This is a real, honest gap to record, not something to fake past.

Upgrade to Professional

1 YOU – BROWSER

Same as Tenant 1's upgrade steps, but click **Upgrade** on the **Professional** row instead. Confirm the invoice shows KES 4,900.

2 YOU – BROWSER

Complete payment the same way, confirm the plan switches.

VERIFY BEFORE MOVING ON

- **Tech — Admin console:** **/admin/tenants** shows Tenant 2 as Professional.
- **You — Browser:** the dashboard subtitle and the billing page both agree on the plan name — check both, not just one.

Telegram Only → Business

The easiest channel to connect for real — genuinely free and instant, no excuse to skip it.

Sign up & verify trial

1 YOU — BROWSER

Repeat sign-up as **Tenant 3 — Telegram Only**, phone e.g. **+254700111003**. Confirm the trial card.

Connect Telegram

1 YOU — TELEGRAM APP

On your own phone or Telegram Desktop, message **@BotFather**, send **/newbot**, follow its prompts, and copy the token it gives you.

How to know it worked: BotFather's final message includes a long token string starting with a number, a colon, then letters and digits.

2 YOU — BROWSER

In P2Less, go to **Channels** → Telegram, paste the token into **Connect bot**, submit.

Expect: the section shows a real connected bot username, no approval wait of any kind.

3 YOU — TELEGRAM APP

From your own Telegram, message the bot something a deterministic path can answer (e.g. a number from its menu) — not a free-form question that would need a real AI call.

How to know it worked: the bot replies within a few seconds with the expected menu response.

Upgrade to Business

1 YOU — BROWSER

Same upgrade flow, **Business** row, confirm invoice KES 19,900, complete payment.

VERIFY BEFORE MOVING ON

- **You — Browser:** go to **/dashboard/conversations**. **Expect:** the real inbound Telegram message you sent actually appears as a Conversation with at least one Message.
- **Tech — Admin console:** **/admin/tenants** shows Tenant 3 as Business, with the correct revenue recorded on **/admin/billing**.

Widget Only → Enterprise

The negotiated-contract tier — admin-assigned, not self-service.

Sign up & verify trial

1 YOU — BROWSER

Repeat sign-up as `Tenant 4 — Widget Only`, phone e.g. `+254700111004`.

Connect the widget

1 YOU — BROWSER

Go to **Channels** → Website widget, create a widget key.

2 YOU — BROWSER

Copy the embed snippet, open `/dashboard/widget`'s own preview (or a blank HTML page pasted with the snippet) and send one deterministic test message.

How to know it worked: the message appears back in the P2Less dashboard's conversation list within a few seconds.

Move to Enterprise

NOT SELF-SERVICE, BY DESIGN

Enterprise is deliberately admin-only — the self-service Upgrade modal correctly refuses it. This is the one tenant whose plan change is done from the admin side, on purpose.

⚙️ TECH — ADMIN CONSOLE

From `/admin/tenants/[Tenant 4's id]`, use the real admin plan-change action to move Tenant 4 onto Enterprise, with a realistic negotiated note.

2 YOU — BROWSER

Reload Tenant 4's own dashboard, confirm the plan name updated.

Expect: the usage card now shows "unlimited"/no ceiling, not a stale trial or Starter view.

VERIFY BEFORE MOVING ON

- **Tech — Admin console:** check for a real audit-log entry for this plan change, attributing it to the admin who made it.
- **You — Browser:** Tenant 4's own billing page correctly reflects Enterprise's postpaid/negotiated framing, not a payable invoice.

What happens when money gets messy

These situations are real, but none of them can be produced by simply clicking through the UI in real time — they're staged directly in Prisma Studio, then observed through the same UI a real customer would see.

A partial payment lands

1 YOU – BROWSER

Pick one tenant with an open invoice (e.g. start a fresh upgrade on Tenant 2 without paying yet). Note the invoice's `payableKes` value shown on screen.

TECH – PRISMA STUDIO

Click the `Payment` table, click **Add record**. Set `invoiceId` to that invoice's id (copy it from the `Invoice` table, matching by tenant and creation time), set `amountKes` to something less than the full amount owed — e.g. half of it — set `status` to whatever value the table uses for "confirmed" (check an existing paid row for the exact spelling), save.

3 YOU – BROWSER

Reopen that tenant's upgrade modal.

How to know it worked: it shows "We received KES X toward this invoice — KES Y still due," and does **not** claim the plan has changed. If it instead shows the plan as upgraded already, that's a real bug — a partial payment must never be treated as a full one.

Paid, but the money hasn't arrived yet

1 YOU – BROWSER

Start a real STK payment and, instead of approving it, just wait past the confirmation window without doing anything on the phone.

Expect: after ~90 seconds, the modal honestly says it's still waiting and offers to try again — never a false "success."

TECH – PRISMA STUDIO

Minutes later, add the real `Payment` row as if it had arrived late (same steps as the partial-payment case above, but for the full amount this time), then, from a terminal in the project folder, run the real settlement function directly — ask your engineer for the exact one-off script command, since it calls internal code rather than clicking a button.

How to know it worked: reload the tenant's dashboard — it should now show the upgraded plan, even though the original request had already timed out on screen. A late arrival must still count.

Payment fails or is cancelled

1

YOU — BROWSER

Start a real STK payment and cancel/decline it on the phone (or simulate a decline).

How to know it worked: "Payment was cancelled or failed. Your plan has not changed" — and reloading the dashboard confirms the plan genuinely didn't move.

A realistic usage history, without any real AI cost

Entirely a Prisma Studio job — there's no honest way to "click" three months into existence. What matters is that what gets inserted is realistic and checkable.

⚙️ TECH — PRISMA STUDIO

For each of Tenants 1–4, click the `UsageEvent` table, **Add record** repeatedly (or, if your Prisma Studio version supports pasting multiple rows, use that) to insert rows dated across a 3-month span, at a volume appropriate to their plan — deliberately including one month where one tenant's volume crosses their real cap. Set each row's `tenantId` and `createdAt` deliberately, in the past.

⚙️ TECH — PRISMA STUDIO

Click the `AiRequestLog` table, insert a handful of rows with realistic `totalTokens` / `costUsd` figures matching the real `ModelPricing` table (check its values first) — never by calling a real AI provider.

3 YOU — BROWSER

For each tenant, send a small number of genuinely real messages over a deterministic path (a menu tap, an OTP entry, a templated lookup) — enough to confirm the real send pipeline works, not enough to represent the full 3 months.

THE REAL QUESTIONS THIS ANSWERS

- **You — Browser:** for the tenant whose volume crossed their cap, does the dashboard actually show them blocked that month, and correctly reset the next?
- **You — Browser:** do the dashboard's message-volume chart and usage numbers match what was actually inserted in Prisma Studio?
- **Tech — Admin console:** does `/admin`'s platform-wide view aggregate all four tenants correctly — do the totals shown there actually add up to what you inserted per tenant?
- **Tech — Prisma Studio + Admin console:** compare the `costUsd` sums in `AiRequestLog` against the real KES revenue shown in `/admin/billing` for the same tenants and period — is revenue genuinely higher than cost, per tenant, and as one platform total?
- **Tech — Admin console:** does the audit trail account for every plan change, payment, and balance movement across the whole period, with the tamper-evident hash chain still holding?

Record every step here

Copy this table (or the one in the Dossier artifact) and fill in a row per step above as it's actually run.

TENANT	STEP	TOOL USED	EXPECTED	ACTUAL	PASS / FAIL
Not started — nothing logged yet.					